



PROCESAMIENTO DE IMÁGENES I

Maximiliano Romano(R-4634/5) – Facundo Molina(M-7128/5)

El código desarrollado detecta y puntúa dados que han sido lanzados en una superficie.

El proceso se divide en dos fases principales:

1. **Extracción de Frames estáticos:** Identifica y guarda los momentos en los videos donde el movimiento se detiene (la tirada de dados termina).

2. **Detección y puntuación de dados:** Analiza los frames guardados para localizar objetos de color rojo (los dados) y luego cuenta los puntos dentro de cada dado detectado.

Objetivo General

El código procesa una lista de archivos de video (`tirada_1.mp4`, `tirada_2.mp4`, etc.) para:

1. Encontrar el momento en que los objetos (dados) están quietos.
2. Guardar esas imágenes (frames estáticos).
3. Detectar los dados de color rojo en la imagen.
4. Contar los puntos de cada dado.
5. Mostrar la imagen resultante con las detecciones y puntuaciones.

I. Configuración Inicial y Parámetros

El código inicializa variables y parámetros clave utilizados a lo largo del script:

- Archivos de video: Se define una lista de rutas a los videos a procesar (`archivos_videos`).
- Directorio de salida: Se asegura la existencia del directorio "`frames/`" para guardar los frames extraídos.
- Parámetros de detección de movimiento:
 - `umbral_mov = 2.0`: Umbral de diferencia media de píxeles entre frames (una puntuación baja indica poco movimiento).
 - `frames_estaticos > 18`: Número de frames consecutivos por debajo del umbral de movimiento, necesario para considerar que el dado está quieto.
- Parámetros de detección de color (rojo): Se definen rangos de color HSV para detectar objetos rojos, combinando dos rangos (`rojo_bajo_1` a `rojo_alto_1` y `rojo_bajo_2` a `rojo_alto_2`) debido a la naturaleza cíclica del rojo en HSV.
- Parámetros de detección de contorno (dado):
 - `AREA_MINIMA = 500`, `AREA_MAXIMA = 30000`: Rango de área para filtrar contornos que son demasiado pequeños o demasiado grandes para ser un dado.
 - `aspect_ratio` (0.5 a 1.5): Filtro de relación de aspecto para asegurar que el objeto sea aproximadamente cuadrado.

II. Fase 1: Extracción de Frames Estáticos

Esta sección itera sobre cada video para encontrar y guardar los frames donde el movimiento se detiene.

1. Captura de video: El código abre cada archivo de video usando `cv2.VideoCapture()`.
2. Preprocesamiento del frame: Cada frame se convierte a escala de grises (`cv2.cvtColor()`) y se le aplica un desenfoque Gaussiano (`cv2.GaussianBlur()`) para reducir el ruido.
3. Detección de movimiento:
 - Calcula la diferencia absoluta (`cv2.absdiff()`) entre el frame actual (desenfocado en gris) y el frame anterior.
 - Calcula la `motion_score` (puntuación de movimiento) como la media de esta diferencia de píxeles.
 - Si el `motion_score` es inferior al `umbral_mov`, incrementa el contador de `frames_estaticos`. Si no, lo reinicia.
4. Guardado del frame estático: Si el contador de `frames_estaticos` supera el umbral (18 o 25 frames), se considera que los dados están quietos. El frame original (BGR) en ese instante se guarda como una imagen JPG dentro de una carpeta específica para cada video (Ej: `frames/tirada_1/frame_0.jpg`).

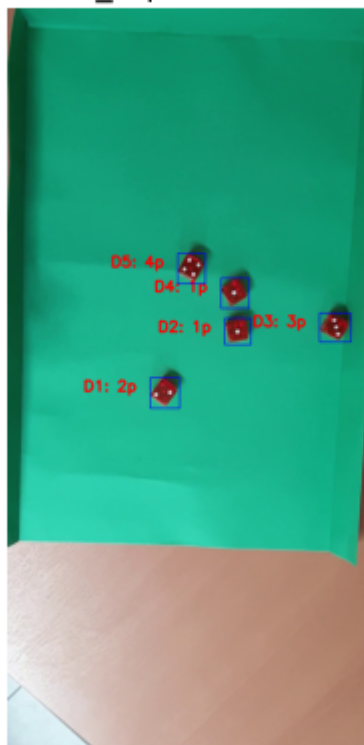
III. Fase 2: Detección y puntuación de dados

Esta sección procesa las imágenes estáticas guardadas para identificar y contar los puntos de los dados.

1. Lectura de frames: Itera sobre todos los archivos JPG guardados en la carpeta de frames para cada video.
2. Segmentación por color (rojo):
 - Convierte la imagen a espacio de color HSV (`cv2.cvtColor()`).
 - Crea una máscara binaria (`imagen_binaria`) aplicando los dos rangos de color rojo definidos con `cv2.inRange()`.
3. Filtrado morfológico: Aplica operaciones morfológicas (cierre, dilatación, apertura) con un kernel de 9x9 para:
 - Rellenar pequeños agujeros dentro de las regiones rojas (cierre).
 - Suavizar los contornos (dilatación/apertura).
4. Detección de contornos del dado:
 - Busca contornos externos (`cv2.findContours()`) en la `imagen_binaria`.
 - Filtra los contornos basándose en `AREA_MINIMA`, `AREA_MAXIMA` y la `aspect_ratio` (relación de aspecto) para identificar únicamente los dados.
5. Análisis de región de interés (ROI) y conteo de puntos:
 - Para cada contorno clasificado como dado, define una Región de Interés (ROI) que encierra el dado.
 - Aplica umbralización adaptativa Gaussiana (`cv2.adaptiveThreshold()`) a la ROI en escala de grises para aislar los puntos del dado, que suelen ser de un color claro sobre el rojo oscuro del dado.
 - Aplica una operación de apertura para limpiar pequeños ruidos de píxeles.
 - Busca nuevos contornos (`cnts_puntos`) dentro de esta región umbralizada.

- Filtra estos contornos internos basándose en el área y la circularidad (calculada a partir del perímetro y el área) para contar cuántos de ellos son puntos reales del dado.
6. Visualización de Resultados:
- Dibuja un rectángulo azul alrededor de cada dado detectado en la imagen.
 - Superpone la puntuación del dado (el número de puntos contados) como texto rojo.
 - Muestra el resultado final.

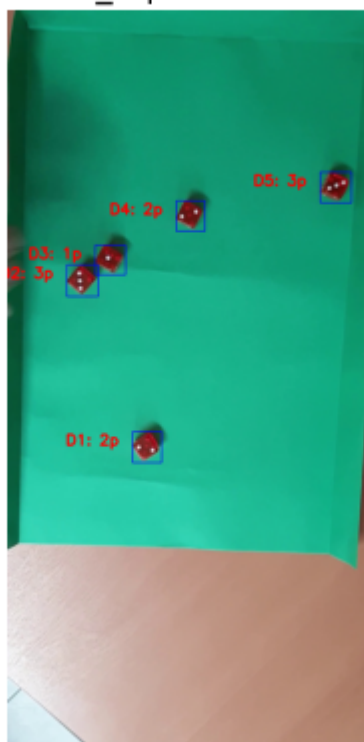
Video: tirada_1 | Frame: frame_1.jpg



Video: tirada_3 | Frame: frame_1.jpg



Video: tirada_2 | Frame: frame_0.jpg



Video: tirada_4 | Frame: frame_10.jpg

